

System Solution Architecture

High Priority Car Rental Management System (HPCRMS)

Prepared by: Orion Blackstock (618684)

Course Project — Software Engineering

GitHub: https://github.com/StockDev136/SWE_Final_Project_HPCRMS

Document Version: 1.0

Date: July 17, 2026

Table of Contents

Table of Contents 2

1. Introduction..... 3

 1.1 Purpose..... 3

 1.2 Scope..... 3

 1.3 Intended Audience and Reading Suggestions 3

 1.4 References..... 3

2. Level System Architecture..... 3

 2.1 Physical Tiers 4

 2.2 Logical Layers 4

 2.3 Key Interactions 5

1. Introduction

1.1 Purpose

The purpose of this document is to demonstrate the architectural analysis and System (Solution) Architecture for the High Priority Car Rental Management System (HPCRMS).

1.2 Scope

This is architecture intended to establish the major tiers, layers, and technology choices for HPCRMS. It covers the client application, the backend service, the data store, and the external systems HPCRMS integrates with.

1.3 Intended Audience and Reading Suggestions

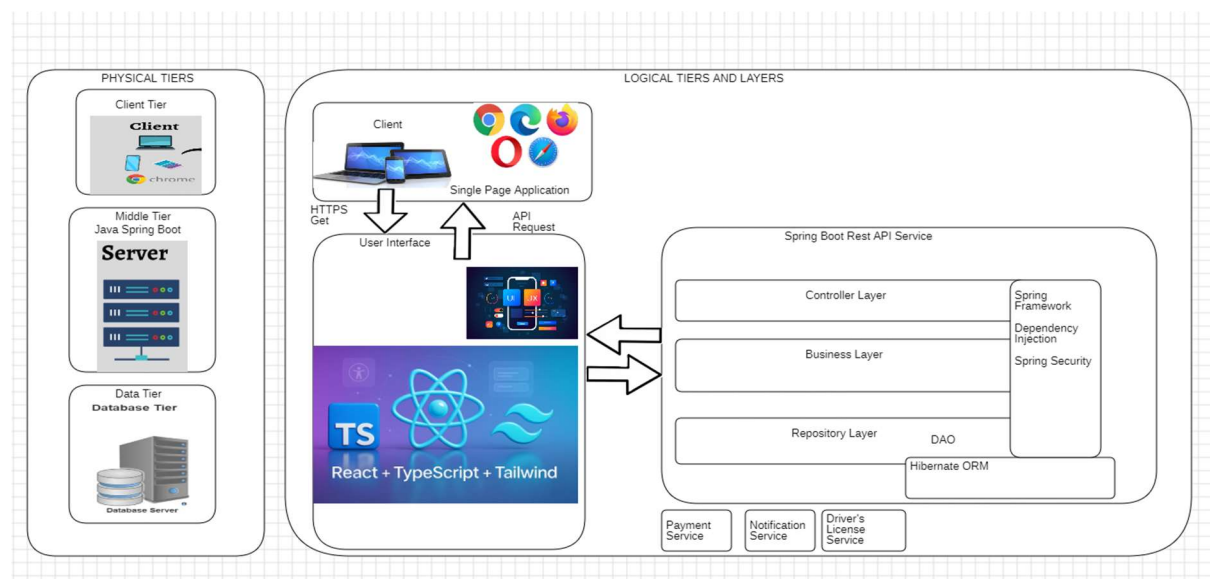
This document is intended for developers, testers, project managers, instructors, and stakeholders. Readers should review Sections 1 and 2 first, then Section 3 for detailed functional requirements and the appendices for analysis models and glossary terms.

1.4 References

- HPCRMS Vision Document
- HPCRMS System Requirements
- Lesson 6 course materials — Architectural Analysis and Design.
- Sample System Architecture diagram

2. Level System Architecture

Figure 1 shows the initial high-level architecture, adapted from the sample architecture diagram provided. It shows the physical tiers (client, application/middle, and data tiers) alongside the logical tiers and layers within the application tier.



2.1 Physical Tiers

Tier	Description
Client Tier	End-user devices (desktop and mobile browsers) running the React single-page application.
Application / Middle Tier	Cloud-hosted application server(s) running the Spring Boot backend. Instances are stateless and run behind a load balancer to meet the 1,000-concurrent-user and 99.9%-availability requirements.
Data Tier	A managed relational database (MySQL or PostgreSQL) plus the external third-party services (payment, license verification, notifications) and cloud object storage for uploaded images.

2.2 Logical Layers

Layer	Responsibility	Technology
Presentation Layer	Renders the UI, manages client-side state and routing, and calls the backend REST API for all data. Holds the authenticated user's JWT for the session.	React, TypeScript, React Router, Axios/Fetch, Tailwind CSS
Controller Layer	Exposes REST endpoints, validates and maps requests/responses (DTOs), and translates HTTP concerns (status codes, headers) to and from the Service layer.	Spring MVC (@RestController)
Service Layer	Implements business rules and orchestrates use cases (e.g., reservation creation, check-in, payment), and defines transaction boundaries.	Spring (@Service, @Transactional)
Repository / DAO Layer	Abstracts data access and query logic over the relational database.	Spring Data JPA
ORM	Maps Java domain objects to relational tables.	Hibernate
Cross-Cutting: Security	Authenticates requests, enforces role-based access control, and issues/validates tokens.	Spring Security, JWT
Data Store	Persists all core application data (customers, reservations, vehicles, payments, maintenance, users).	MySQL / PostgreSQL
External Integrations	Outbound calls to third-party systems, isolated from core business logic.	REST clients to Payment Gateway, License Verification Service, Notification Service

2.3 Key Interactions

- The browser loads the React SPA once (HTTPS GET); thereafter the SPA communicates with the backend exclusively through asynchronous REST calls returning JSON.
- Every request from the Presentation layer passes through Spring Security at the Controller layer, which validates the caller's JWT and role before the request reaches the Service layer.
- The Service layer is the only layer permitted to call external integrations (Payment Gateway, License Verification, Notification) and the only layer that starts and commits transactions.
- The Repository layer is the only layer that issues queries against the database, keeping data access logic out of the Controller and Presentation layers.